The `Assert.Throws` method executes the lambda and catches the exception. If the exception thrown matches the expected type, the test passes. If no exception is thrown or the exception thrown isn't of the expected type, the `Assert.Throws` method throws an exception and fails the test.

That brings us to the end of this brief introduction to unit testing with xUnit. The examples in this section described how to use the new .NET Test SDK, but we didn't cover anything specific to ASP.NET Core. In chapter 36 we'll focus on applying these techniques to testing ASP.NET Core projects specifically.

Summary

- Unit test apps are console apps that have a dependency on the .NET Test SDK, a test framework such as xUnit, MSTest, or NUnit, and a test runner adapter. You can run the tests in a test project by calling `dotnet test` from the command line in your test project or by using Test Explorer in Visual Studio.
- Many testing frameworks are compatible with the .NET Test SDK, but xUnit has emerged as an almost de facto standard for ASP.NET Core projects. The ASP.NET Core team themselves use it to test the framework.
- To create an xUnit test project, choose xUnit Test Project in Visual Studio or use the `dotnet new xunit` CLI command. This creates a test project containing the Microsoft.NET.Test.Sdk, xunit, and xunit.runner.visualstudio NuGet packages.